haz3-jolt / Hari Srinivasan

GitHub: haz3-jolt
Discord: haz3-jolt

AnkiDroid GSOC 2025 Proposal

Tablet & Chromebook UI

haz3-jolt / Hari Srinivasan **1**
Requirements **3**
By submitting this GSoC proposal, I confirm that I: 3
My contributions to AnkiDroid: 3
Tablet & Chromebook UI (175 Hours): **4**
Problem 4
I want to work on it because: 4
How I will solve it/make progress: 4
1. Resizable Layouts 4
2. Add Previewer to NoteEditor 9
3. Togglable Hints 11
4. Mouse Improvements 16
5. Optional Extensions 20
Timeline 25
Myself: **27**
My personal experience with Anki/AnkiDroid: 27
What I expect to get from participating in GSoC 27
Programming languages and tools I know 27
Example of skills you taught yourself 27
Anything else: **28**

Requirements

By submitting this GSoC proposal, I confirm that I:

[✅] Meet Google's Eligibility Requirements
[✅] Have installed and used AnkiDroid
[✅] Have a basic understanding of Anki's concepts such as decks, cards, notes, note types, new cards, review cards and cards in learning
[✅] Have access to a physical Android device
[✅] Have access to an internet connection throughout GSoC. At least enough to communicate via text, update Android Studio when required, and interact with GitHub
[✅] Have at least 175 hours to dedicate to the GSoC, and have time for at least one one-to-one meeting with a mentor each working week, to discuss progress and future tasks
[✅] Have been able to compile AnkiDroid, install it on a device, run automated tests and enable WebView Debugging
[✅] Have sufficient English reading & writing abilities to participate in technical discussions
[✅] Have some experience with Kotlin programming, testing, debugging, and git

My contributions to AnkiDroid:

- Link to a merged AnkiDroid Pull Request (PR) I created that contains at least one unit test[1] :
  - https://github.com/ankidroid/Anki-Android/pull/17795
  - https://github.com/ankidroid/Anki-Android/pull/17680
  - https://github.com/ankidroid/Anki-Android/pull/16188
  - https://github.com/ankidroid/Anki-Android/pull/16307
- Link to a merged AnkiDroid PR I created that contains at least one code change that is not a test:
  - https://github.com/ankidroid/Anki-Android/pull/16472
  - https://github.com/ankidroid/Anki-Android/pull/17795
  - https://github.com/ankidroid/Anki-Android/pull/17616
  - https://github.com/ankidroid/Anki-Android/pull/17626
- Link to an AnkiDroid PR I constructively[2] commented on that I did not create:
  - https://github.com/ankidroid/Anki-Android/pull/18052#issuecomment-2701383004

[1] Look at https://github.com/ankidroid/Anki-Android/wiki/Testing if you are new to unit tests.
[2] A comment who helped improve the PR. E.g. stating that something is not clear and requires comments/javadoc/detailed commit message, suggesting an improvement to the change,

Tablet & Chromebook UI (175 Hours):

Problem

AnkiDroid was initially designed for Android mobile phones. Over the years, Android has come to tablets and Chromebooks, but our UI has continued to be designed around the mobile phone.

Sanjay Sargram's project last year has *massively* improved the UI, there are still a few places where improvements can be made.

I want to work on it because:

Lately, I've been using AnkiDroid on my tablet much more, especially with a keyboard case, and I've noticed how certain workflows still feel clunky. As a moderate power user, it can be frustrating to constantly switch between the note editor and the preview page or to look up shortcuts just to navigate efficiently.

Beyond personal interest, improving the tablet and Chromebook experience is a great way to give back to the community which got me into anki, students. Many students rely on school-issued Chromebooks for studying, and a more optimized UI could make their life a ton easier. I'm also looking forward to the challenge of making the UI work better on bigger screens and figuring out the quirks that come with it.

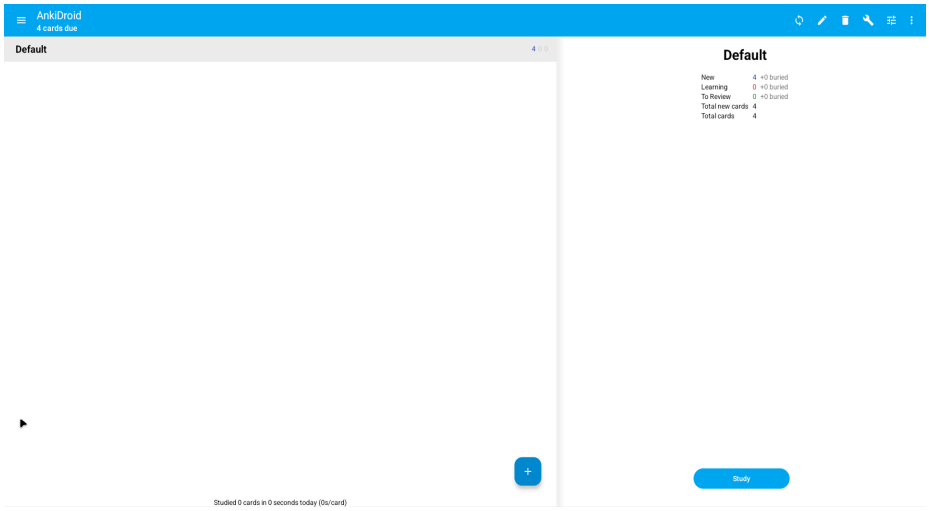
How I will solve it/make progress:

1. **Resizable Layouts**

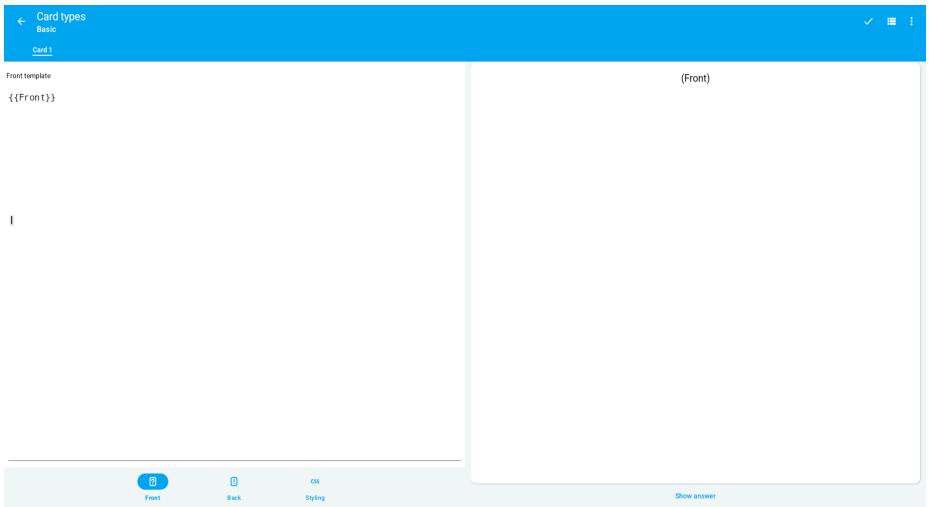
   The current Deckpicker and CardTemplateEditor have two panes but it is not possible to resize them, this will also apply for the CardBrowser once #16794 is merged.
   Allowing users to resize the panes will give them a more desktop-esque experience by allowing them to customize the exact look of their AnkiDroid setup.

   Note: As of writing this proposal, #16794 is yet to be merged, so the images for the CardBrowser might be slightly outdated.

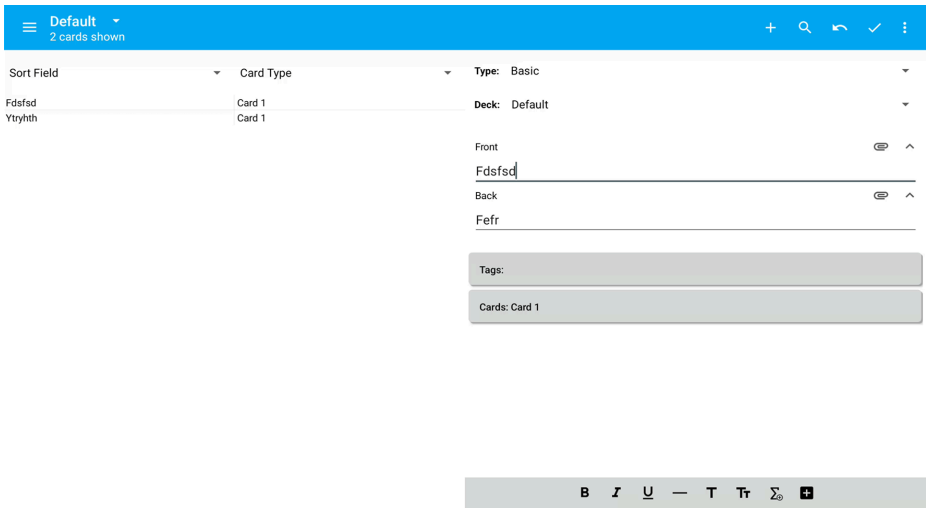
Current Screens:



DeckPicker



CardTemplateEditor



CardBrowser

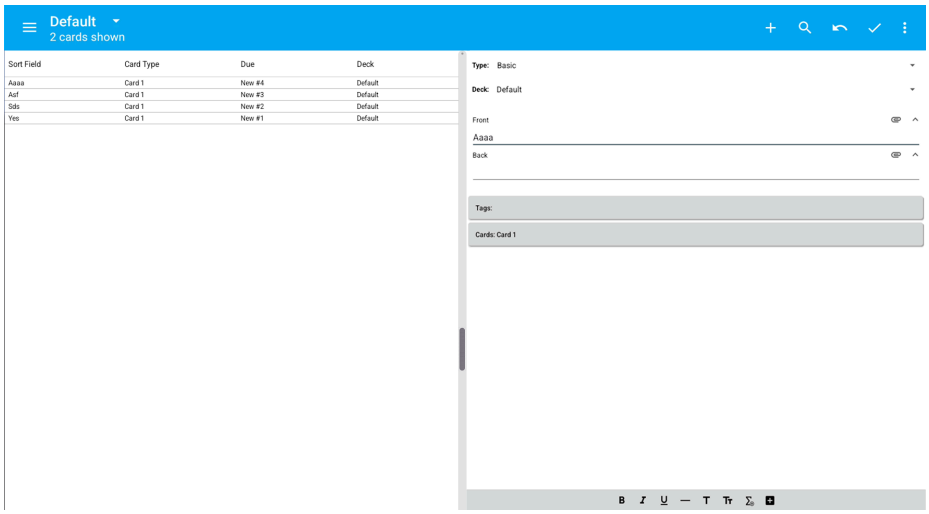
Solution:

We can solve this by introducing a draggable divider between the two panes, allowing users to adjust their sizes dynamically. This will be implemented using a simple View as a separator, which will listen for touch gestures and adjust the layout weight of the surrounding views accordingly. This will work when the layout is in fragmented mode (side-by-side view on larger screens).

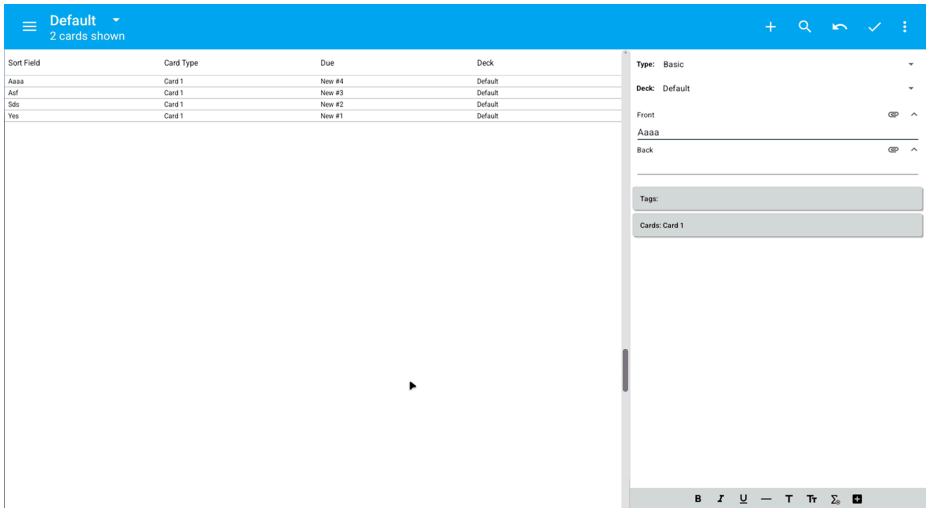
To ensure a smooth user experience, we will:

- Add visual cues, such as a grippy handle or a rounded rectangle on the divider, to indicate that it is draggable.
- Implement constraints to prevent extreme resizing (e.g., minimum and maximum width percentages).
- Store user preferences, so the adjusted layout persists across app restarts.

Example:



Default



Resized

Implementation:

XML Mockup:

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
      xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="match_parent"
      android:layout_height="match_parent">

      <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:orientation="vertical">

      <include layout="@layout/toolbar" />

      <LinearLayout
            android:id="@+id/card_browser_xl_view"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:background="?android:attr/colorBackground"
            android:orientation="horizontal">

            <!-- Card Browser -->
            <include layout="@layout/cardbrowser"
                  android:id="@+id/card_browser"
                  android:layout_width="0dp"
                  android:layout_weight="3"
```

```
                  android:layout_height="match_parent" />

            <!-- Draggable Divider -->
            <View
                  android:id="@+id/draggable_divider"
                  android:layout_width="8dp"
                  android:layout_height="match_parent"
                  android:background="@drawable/draggable_handle"
/>

            <!-- Note Editor -->
            <androidx.fragment.app.FragmentContainerView
                  android:id="@+id/note_editor_frame"
                  android:layout_width="0dp"
                  android:layout_weight="2"
                  android:layout_height="match_parent" />

      </LinearLayout>
      </LinearLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

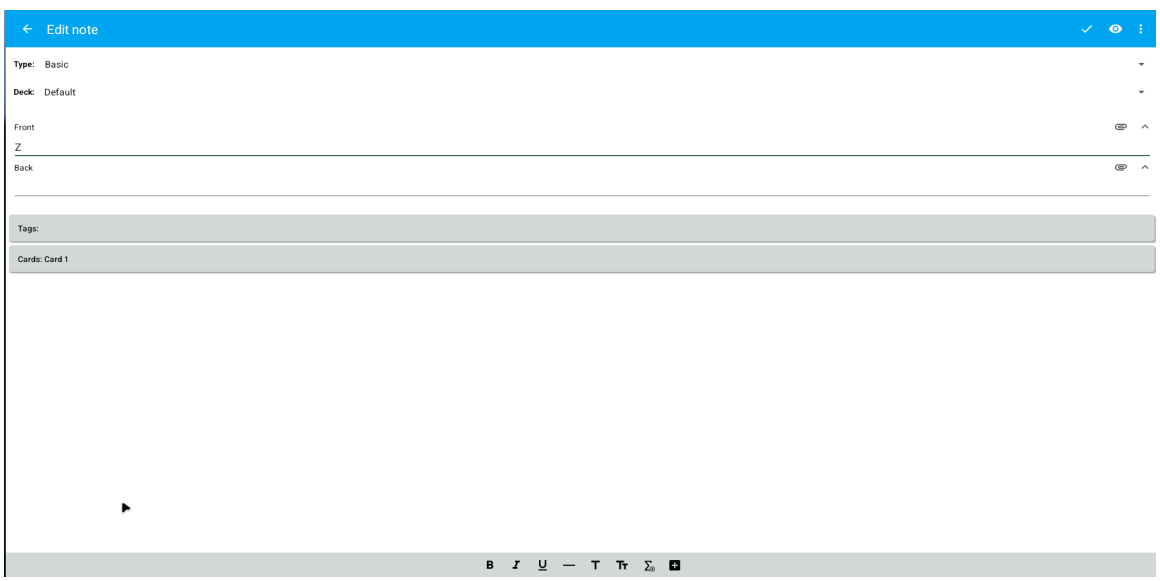
Kotlin implementation snippet:

```
      private var startX = 0f
      private var startWeightLeft = 0f
      private var totalWeight = 5f

      override fun onFinishInflate() {
      super.onFinishInflate()

      val leftPane = findViewById<View>(R.id.card_browser)
      val divider = findViewById<View>(R.id.draggable_divider)
      val rightPane = findViewById<View>(R.id.note_editor_frame)

      divider.setOnTouchListener { _, event ->
            when (event.action) {
                  MotionEvent.ACTION_DOWN -> {
                  startX = event.rawX
                  startWeightLeft = (leftPane.layoutParams as
LinearLayout.LayoutParams).weight
                  }

                  MotionEvent.ACTION_MOVE -> {
```

```
                    val deltaX = event.rawX - startX
                    val newWeightLeft = (startWeightLeft + deltaX /
width * totalWeight).coerceIn(1f, totalWeight - 1f)

                    (leftPane.layoutParams as
LinearLayout.LayoutParams).weight = newWeightLeft
                    (rightPane.layoutParams as
LinearLayout.LayoutParams).weight = totalWeight - newWeightLeft

                    leftPane.requestLayout()
                    rightPane.requestLayout()
                    }
                }
                true
            }
}
```

To ensure the user's preferred layout persists across sessions, we can store the adjusted weights using SharedPreferences. When the app restarts, we retrieve and apply these values to maintain the customized experience.

2. Add Previewer to NoteEditor



Current Note Editor View

The proposed change introduces a dual-pane layout for the Note Editor, integrating a live previewer. Users will be able to edit notes while simultaneously seeing a real-time preview of the generated cards.

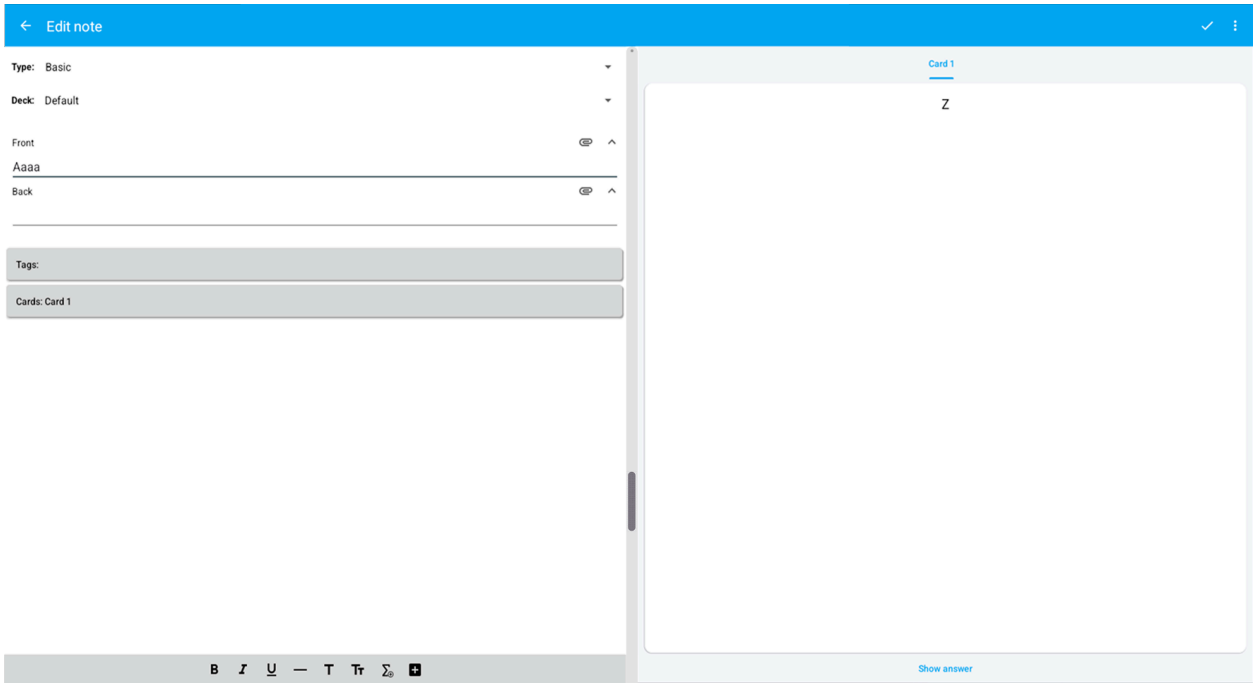
This will be implemented as part of the layout-xlarge group, building upon the structured approach used in previous large-screen UI improvements. Similar to how past updates streamlined the interface for larger displays, this feature will ensure consistency across different screen sizes. The Previewer will automatically refresh after a short delay (e.g., 1 second) when the user finishes editing, mirroring Anki's behavior.

Since this will be developed after the resizable layout feature, the same resizable layout will be applied here, allowing users to adjust the editor-preview ratio dynamically.

If the current Note Editor is replaced by the new Svelte-based implementation, necessary adjustments will be made to integrate the Previewer with it.

This feature also has the really nice side effect of letting users who are new to anki to intuitively understand that some notes generate multiple cards, and how cloze deletions can rapidly create many cards from a single note, helping partially complete the goals of the Note Type Preview project.

All the menu item actions for the Note Editor will be available in the new layout under the 'More options' menu. These actions will be handled directly by the Note Editor, ensuring that its functionality remains self-contained and consistent.



Noteditor and Previewer coupled together

Implementation:
Instead of reloading a new WebView each time (which would be inefficient), we can dynamically update the WebView content when the note editor content changes, with a 1-second debounce delay to avoid excessive updates. This approach ensures smooth updates without unnecessary reloading.

For JavaScript handling, by default, JS execution will be disabled to prevent unintended interactions or blocking alerts. If a note contains JavaScript:

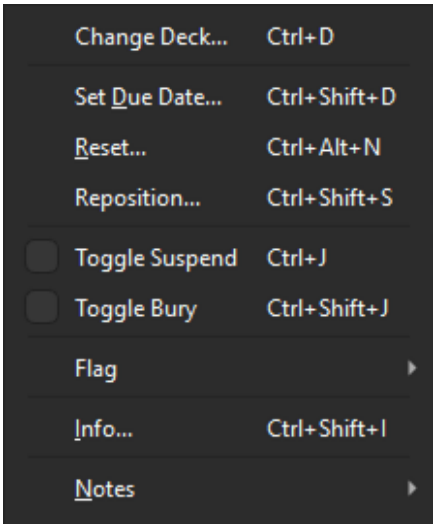
- On first-time opening of the Note Editor + Preview mode, we will show a one-time alert informing the user that JavaScript execution is disabled in this mode.
- A system toggle will be available for advanced users to manually enable JavaScript execution.
- If JavaScript is enabled, note rendering in the editor-preview mode will be disabled to prevent execution loops (e.g., blocking modals from alert()).
- This preference will be stored, ensuring the user is not repeatedly shown the same alert.

To implement this, we will wrap the original note editor's XML within the resizable layout XML (as done in other x-large versions). The Previewer will be integrated using a FragmentViewContainer, ensuring proper separation while maintaining flexibility in layout adjustments.

3. Togglable Hints

Menu Hints:

Currently, AnkiDroid has a shortcut dialog that displays all available keyboard shortcuts, introduced in a PR last year. However, there can be inherent friction to being aware of this to users who do not poke around ankidroid. This proposal enhances shortcut visibility by integrating them directly into the UI in a chip-style format—similar to how shortcuts appear in the right-click menu of desktop applications.



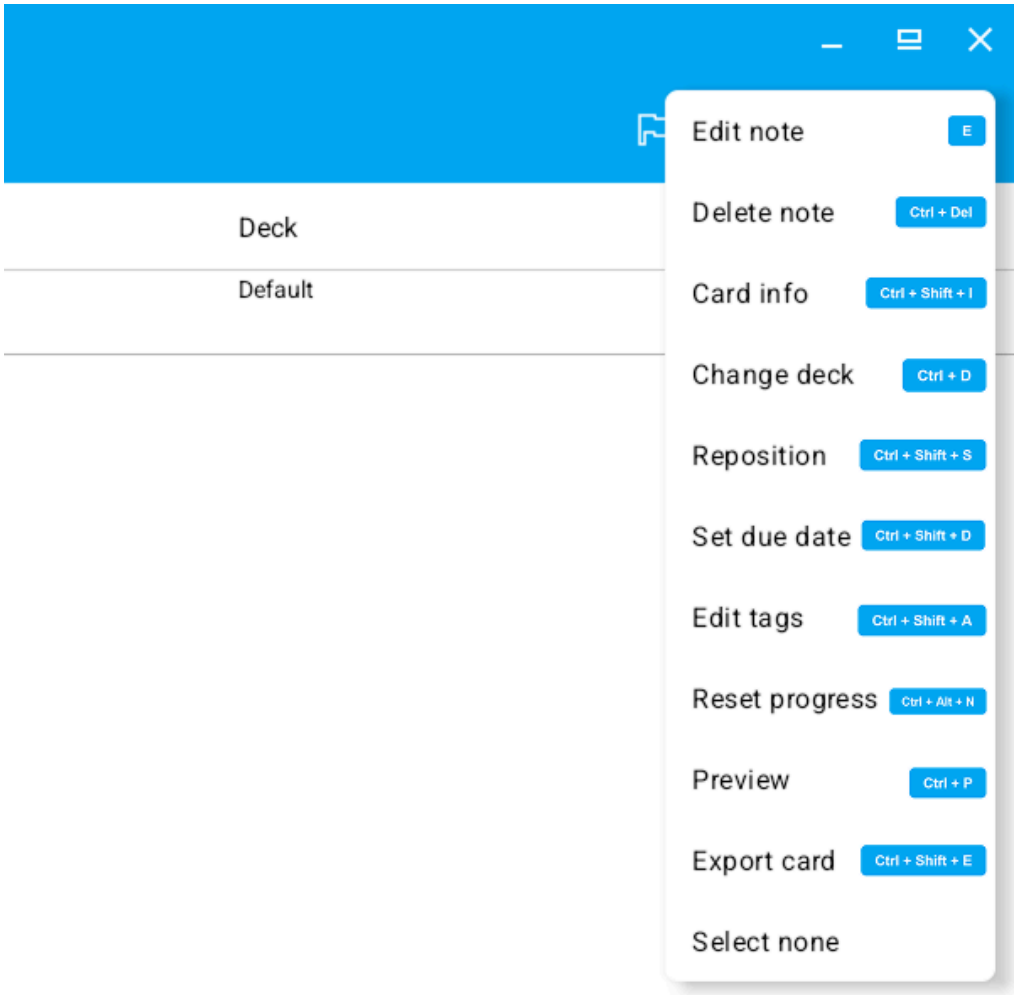
In anki

With this change, users will have the option to toggle persistent shortcut visibility next to relevant buttons in a dropdown or action bar. Instead of overcrowding the UI with text-based shortcuts, they will be formatted as compact, visually distinct chips for an easily readable look.

Key benefits:

- Reduces friction in learning shortcuts by making them immediately visible instead of hidden behind a dialog.
- Improves discoverability of keyboard shortcuts, helping users take advantage of them more effectively.
- Maintains a clean UI with minimal crowding issues by using a chip-based format.

Since this will be a toggleable setting, users who prefer the existing shortcut dialog or do not use shortcuts can disable the chips entirely in settings.



Menu Dropdown With chip style shortcuts

Implementation:

We will use Material UI Chips within a custom view to display keyboard shortcuts alongside menu options.

- The entire shortcut (e.g: "Ctrl+D") will be represented as a single chip to maintain a compact layout.
- On large screens, shortcuts will be positioned horizontally beside menu options to prevent unnecessary scrolling.
- We will detect physical keyboard presence using InputDevice.getDeviceIds() and only show shortcuts when a hardware keyboard is connected.
- Introduce a toggle setting for those who prefer to not see shortcuts even with the keyboard.

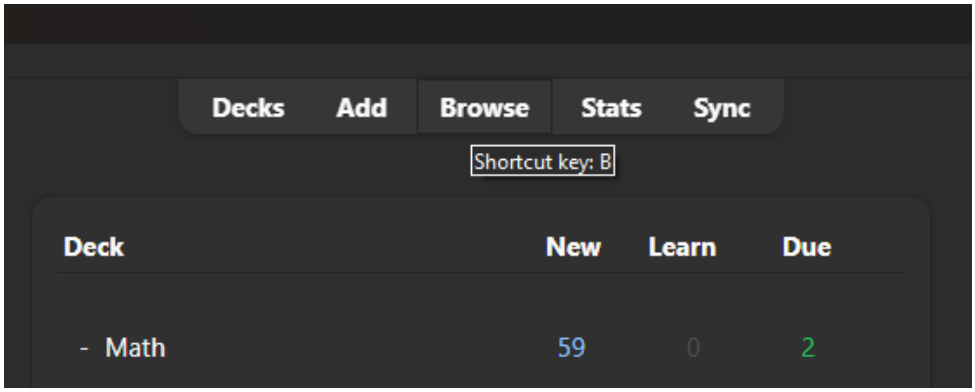
```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:orientation="horizontal"
      android:padding="8dp">

      <com.google.android.material.textview.MaterialTextView
      android:id="@+id/menu_text"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Undo"
      android:textSize="16sp"
      android:textColor="?attr/colorOnSurface" />

      <com.google.android.material.chip.Chip
      android:id="@+id/menu_shortcut"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Ctrl+Z"
      style="@style/Widget.MaterialComponents.Chip.Action"/>
</LinearLayout>
```

Keyboard Detection:

```
fun isHardwareKeyboardConnected(): Boolean {
      return InputDevice.getDeviceIds().any { id ->
      val device = InputDevice.getDevice(id)
      device.keyboardType ==
InputDevice.KEYBOARD_TYPE_ALPHABETIC
      }
}
```

On Hover Hints:

Some shortcut hints on Anki appear when a user hovers over an **option**, or app bar icon appearing as a brief tooltip.



A shortcut key tooltip in Anki

This can easily be implemented with an onHover listener to display a tooltip from TooltipCompat for all options with a shortcut like deckpicker, add note, card browser, etc.

Mockup of a shortcut tooltip

Implementation:

We will use the [TooltipCompat]() library because until API level 26 we had to use it to emulate setTooltipText, we already use it in our [codebase]().

```
import android.view.MotionEvent
import android.view.View
import androidx.appcompat.widget.TooltipCompat

fun View.enableHoverTooltip(tooltipText: String) {
      setOnHoverListener { view, event ->
```

```
      if (event.action == MotionEvent.ACTION_HOVER_ENTER) {
            TooltipCompat.setTooltipText(view, tooltipText)
      }
      false
      }
}
```

Keyboard Detection:

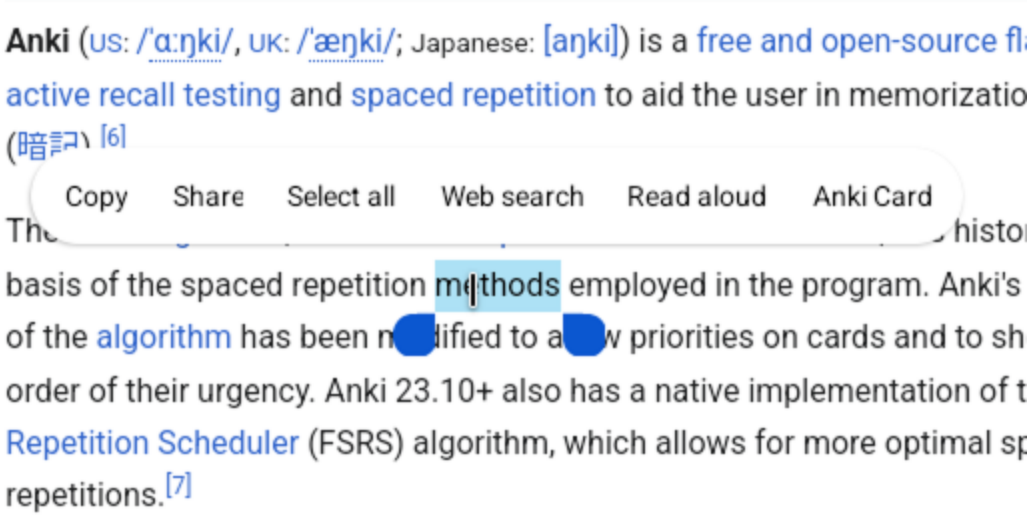
```
fun isHardwareKeyboardConnected(): Boolean {
      return InputDevice.getDeviceIds().any { id ->
      val device = InputDevice.getDevice(id)
      device.keyboardType ==
InputDevice.KEYBOARD_TYPE_ALPHABETIC
      }
}
```

4. Mouse Improvements

Right Click context menus:

Right-click context menus are a common UX pattern that allow users to quickly access relevant actions without navigating through additional menus. Sanjay's work last year allowed us to open dialogs this way, but letting us perform context actions in the cardBrowser or deckpicker with floating context menus would be helpful instead of having to always use the options menu.

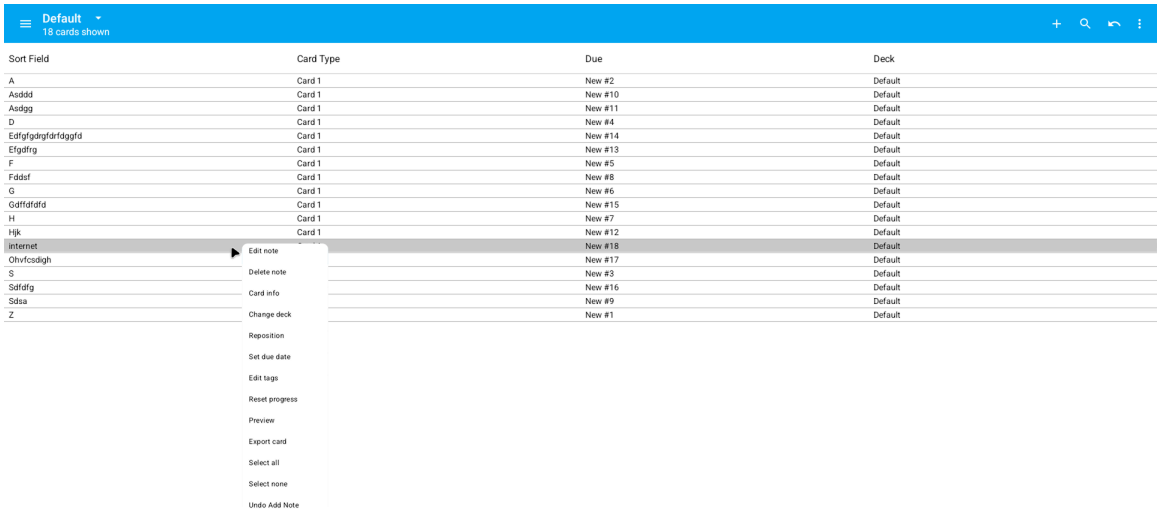
For example, in Chrome, right-clicking on a web page brings up a context menu with options like "Copy," "Share," and "Anki Card," providing immediate access to commonly used features.



An example of a right-click menu on a webpage in Chrome

In AnkiDroid, this feature would enhance usability by introducing right-click menus in areas like the Card Browser, Deck Picker, and Note Editor. Instead of requiring users to tap the “more options” menu, they could simply right-click to bring up a menu with actions present in the “more options” menu. For example, in the Card Browser, right-clicking a single card could present options present in the options menu like "Edit Note", "Delete Note", "Card Info", etc; including all relevant options already present in the options menu. This would also give a more desktop feel to larger devices.

Note: Didn’t add in the mockup, but the right-click context menu will also contain the same shortcut chips as here.



A mockup of the proposed solution

Implementation:

```
fun View.setupRightClickMenu(context: Context) {
      setOnTouchListener { _, event ->
      if (event.action == MotionEvent.ACTION_DOWN &&
event.buttonState == MotionEvent.BUTTON_SECONDARY) {
            showContextMenu(context, this, event.rawX,
event.rawY)
            true
      } else {
                  false
      }
      }
}
```

Right Click Selection in Card Browser:

Currently, dialogs that require a long press also support right-clicking in the deck picker and note editor. This behavior can be extended to the card browser, allowing right-click to select a card, just like long-press does on touch devices. Since we are already implementing a right-click context menu, this change can be clubbed with the previous one, meaning the default behavior when you right-click a card:

Right-click action flow:

1. User right-clicks a card
2. The card gets selected
3. The context menu opens for available actions to perform

This improves parity with Anki Desktop, providing a more familiar desktop-like experience on larger devices like Chromebooks.

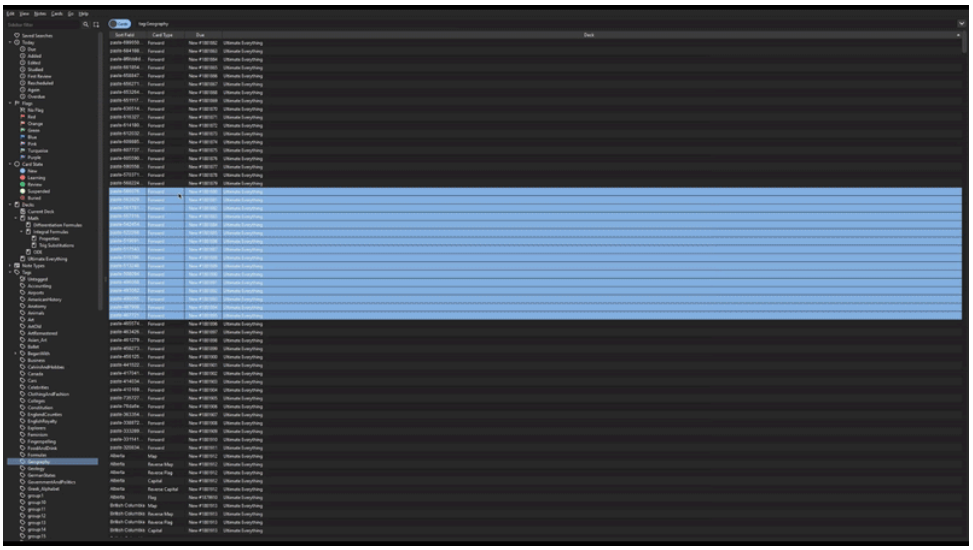
Range Selection in Card Browser:

Currently, selecting multiple cards in AnkiDroid requires manually tapping each one. This can be improved by adding Shift + Click support, allowing users to quickly select a range of cards, just like in Anki Desktop.

Behavior:

1. User selects a card.
2. User holds Shift and clicks another card.
3. All cards between the first and second selection are highlighted.

This provides a faster and more intuitive selection method, especially for Chromebook and tablet users who have mouse input.



Range Selection in Anki

Implementation:

We will use the [RecyclerView Selection library]() to manage selection state. Since the library now includes support for band selection via the [BandPredicate]() class, we can utilize this to implement Shift + Click range selection. If the default behavior is insufficient for our specific needs, we can extend the BandPredicate to customize how ranges are selected.

Behaviour:

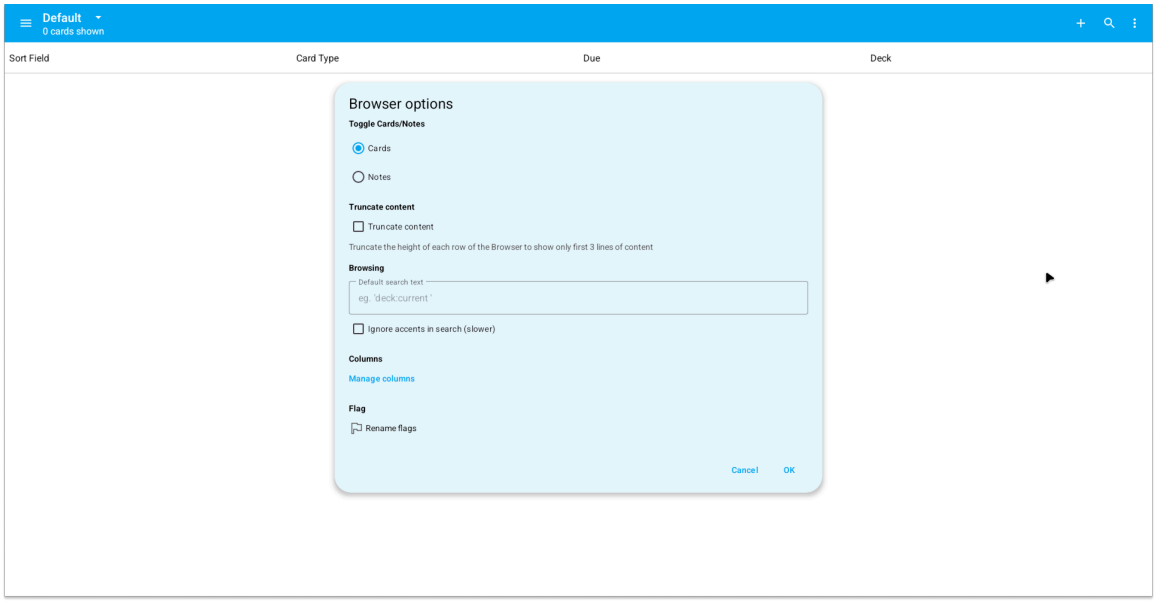
- When the user Shift + Clicks:
  - Track the first and last clicked positions.
  - Update the selection state for all items in between.
- When the user clicks an already selected item within the range:
  - Remove that item from the selection state.
  - Preserve the rest of the selected range.

5. Optional Extensions

These are minor features I plan on including if time permits. Features marked as **(nice to have)** will have further discussions design/spec wise before being finalized.

Notes/Cards Toggle (Nice to Have):

Currently on the CardBrowser screen we have to navigate to the options menu in order to switch between cards and notes:



With the much larger screen real estate on larger devices, we can have greater parity with the desktop version by adding a switch to swap between them without having to dig through the options menu.

Default
10 cards shown
Cards

Sort Field	Card Type
1	Card 1
2	Card 1
3	Card 1
Asd	Card 1
D	Card 1
Dfasd	Card 1
Ssda	Card 1
Ssda	Card 2
Wqqwqwq	Card 1
Wqqwqwq	Card 2

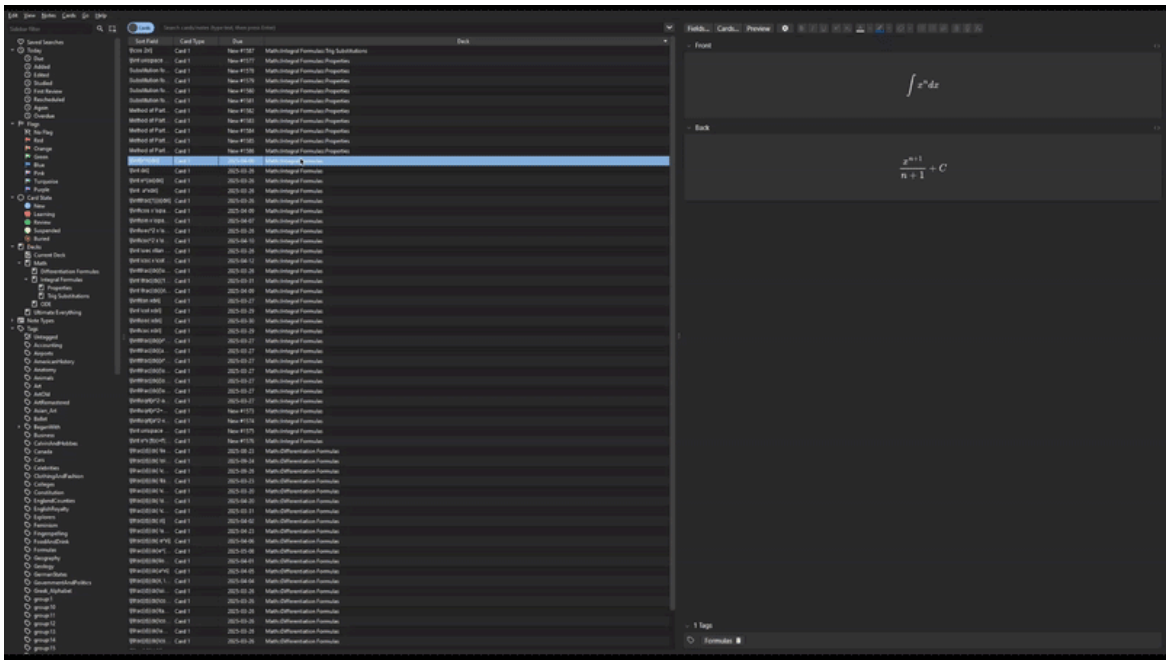
Default
8 notes shown
Notes

Sort Field	Note Type
1	Basic
2	Basic
3	Basic
Asd	Basic
D	Basic
Dfasd	Basic (and reversed card)
Ssda	Basic (and reversed card)
Wqqwqwq	Basic (and reversed card)

Implementation:
We can abstract the current logic for switching between states, and implement a materialUI switch with custom text on the inside to show the current state as Notes or Cards.

Drag Selection (Mouse):

Currently, selecting multiple cards requires individually clicking or once implemented, Shift+Clicking. We can add drag-to-select behavior, where users can click and drag across items to select multiple cards in a fluid gesture, similar to how Gmail or file explorers behave.



Drag selection in Anki Desktop

Behavior:

- User clicks and holds on a card
- Without releasing the mouse, they drag the cursor up/down
- As the cursor moves over cards, they are added to the selection
- Dragging back over already-selected cards deselects them
- Dragging would also auto scroll down or up once it crosses boundary limits.

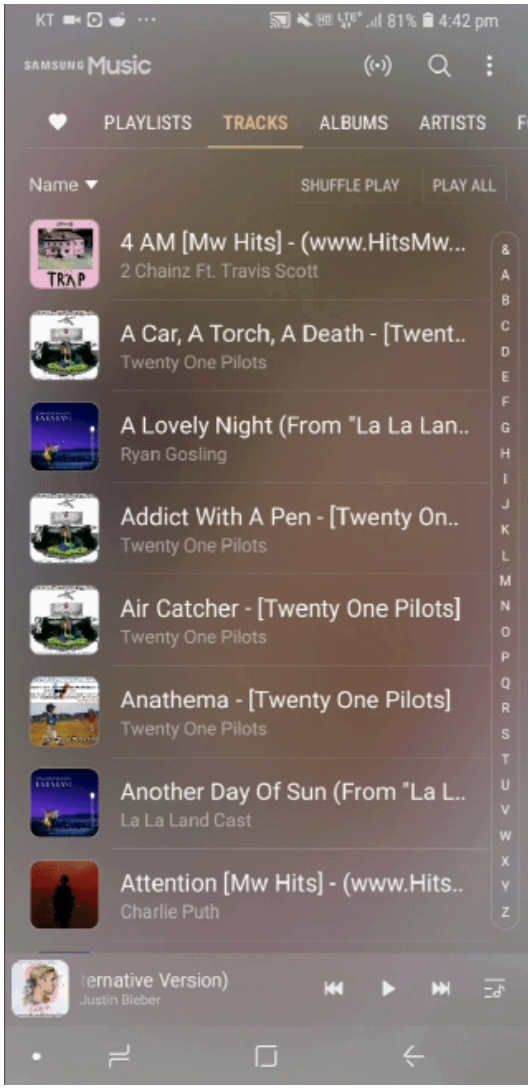
Implementation:

```
var isDragging = false
val selectedPositions = mutableSetOf<Int>()

recyclerView.setOnTouchListener { _, event ->
      when (event.actionMasked) {
      MotionEvent.ACTION_DOWN -> {
            isDragging = true
            val position = getItemPositionUnder(event)
            position?.let { selectItem(it) }
      }
      MotionEvent.ACTION_MOVE -> {
            if (isDragging) {
                  val position = getItemPositionUnder(event)
                  if (position != null &&
!selectedPositions.contains(position)) {
                  selectItem(position)
                  selectedPositions.add(position)
                  }
            }
      }
      MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> {
            isDragging = false
            selectedPositions.clear()
      }
      }
      true
}

// Helper to get adapter position under the touch
fun getItemPositionUnder(event: MotionEvent): Int? {
```

```
      val view = recyclerView.findChildViewUnder(event.x,
event.y)
      return view?.let {
recyclerView.getChildAdapterPosition(it) }
}
```

Drag Selection (Touch):

On touch devices, selecting multiple cards currently involves tapping them one-by-one, which can be slow. By adding a touch-aware drag selection feature, similar to Google Photos or Gmail app, that activates after a long press selects the first item. This prevents accidental selection while scrolling.



Drag selection on phones

Behavior:

- User long-presses on a card to initiate selection (as currently implemented).
- Without lifting their finger, they drag over additional cards to select them.
- Cards under the drag path are toggled:
- If unselected, they become selected.
- If already selected, dragging over them deselects them.
- Scrolling activates automatically if the drag reaches the top/bottom boundary.

This gesture enables quick and intuitive multi-select on phones and tablets without requiring a keyboard.

Implementation:

This version is pretty similar to the mouse one, and also ensures drag selection is only triggered after a long-press, avoiding conflict with scroll gestures or accidental touches.

```
var isDragging = false
val touchedPositions = mutableSetOf<Int>()

recyclerView.setOnItemLongClickListener { _, _, position, _ ->
      isDragging = true
      toggleSelection(position)
      touchedPositions.add(position)
      true
}

recyclerView.setOnTouchListener { _, event ->
      when (event.actionMasked) {
      MotionEvent.ACTION_MOVE -> {
            if (isDragging) {
                  val position = getItemPositionUnder(event)
                  if (position != null &&
!touchedPositions.contains(position)) {
                  toggleSelection(position)
                  touchedPositions.add(position)
                  }
```

```
            }
        }
        MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> {
            isDragging = false
            touchedPositions.clear()
        }
        }
        false
}

fun getItemPositionUnder(event: MotionEvent): Int? {
        val view = recyclerView.findChildViewUnder(event.x,
event.y)
        return view?.let {
recyclerView.getChildAdapterPosition(it) }
}
```

Timeline

Phase	Time Period	Details	Est. Hours
N/A	May 06 - May 20	**University Exams**	N/A
Community Bonding Period	May 20 - June 01	Initial Preparation • Creating XML mockups for Note editor x-large to accommodate Previewer. • Finalise Success criteria for the project • Familiarising myself with the relevant parts of the codebase	N/A
Phase 1	Week 1 June 02 - June 09	Convert DeckPicker to have a resizable Layout • Create a xml layout design • Ensure smooth transitions when resizing • Implement constraints to prevent extreme	25

		resizing ● Persist layout changes across app restarts	
	Week 2 June 10 - June 16	Convert CardTemplateEditor to have a resizable Layout ● Create an XML layout design ● Ensure smooth transitions when resizing ● Persist layout changes across app restarts	10
	Week 3 June 17 - June 23	Convert Note Editor to have a resizable Layout ● Create an XML layout design ● Ensure smooth transitions when resizing ● Persist layout changes across app restarts	10
	Week 4 June 24 - June 30	Add Previewer to NoteEditor ● Create an XML layout design ● Create a NoteEditor layout (x-large group)	15
	Week 5 July 01 - July 07	Add Previewer to NoteEditor ● Include FragmentContainerView ● Implement Resizable layout	20
	Week 6 July 08 - July 11	Buffer period for changes	15
July 12 - July 14		**Submit Midterm Evaluation**	
Phase 2	Week 7 July 14 - July 20	Togglable Hints ● Implement Menu Shortcut Hints ● Implement Keyboard Detection ● Add toggle in settings	15
	Week 8 July 21 - July 27	Togglable Hints ● Implement OnHover Shortcut Hints ● Implement Keyboard Detection ● Add toggle in settings	10
	Week 9 July 28 - July 31	Mouse Improvements ● Right Click to Select ● Band selection in CardBrowser	10
	Week 9 August 01 - August 03	Mouse Improvements ● Right Click Context Menus	15
	Week 10 August 04 -	Optional Extensions	

	August 10		
	August 11 - August 25	Buffer Period	30
August 25 - September 1		**Submit Final Evaluation**	175

Myself:

My personal experience with Anki/AnkiDroid:

I first discovered AnkiDroid while studying for organic chemistry in my sophomore year of high school. It drastically improved my learning speed, allowing me to spend less time on rote memorization while still improving my grades. While I no longer rely on it as much, Anki remains a key part of my learning routine with the Anki Ultimate Everything deck, a nice way to brush up on my trivia.

Contributing to AnkiDroid has been incredibly rewarding—I've not only honed my development skills but also had the chance to give back to a project that has significantly impacted my own learning journey.

What I expect to get from participating in GSoC

Through Google Summer of Code, I hope to better my understanding of Android development, particularly in optimizing apps for larger devices like tablets and Chromebooks, an area I haven't touched before. More importantly, I want to make lasting contributions that enhance the user experience for AnkiDroid's community, ensuring that others can benefit from it just as I have.

Programming languages and tools I know

Languages: Kotlin, Python, C#, Js

Tools: Gradle, XML, Git, HTML/CSS

Example of skills you taught yourself

A new skill I picked up while working on AnkiDroid were CI/CD pipelines. I got hands-on experience tweaking APK naming in releases.sh, which meant diving into Bash scripting and learning how automated build processes actually work behind the scenes (& how easy they are to break :).

I also learned how to write proper unit tests. Before working on a few on AnkiDroid, I had never really written a unit test, but I quickly learned that while they can be a pain to write at times, it's often one of the best ways to explore a codebase and get a deep understanding of the function being tested.

Another area where I improved a lot was XML layouts. I was more comfortable with Jetpack Compose before, so working with traditional XML felt clunky at first. But over time, I figured out how to structure layouts properly, and also learned more about Material UI compliance.

Anything else:

I'm genuinely excited about the chance to apply for GSoC with AnkiDroid, not just because it's a great learning opportunity, but because this project has personally impacted my life. What started as a simple attempt to learn Android development for fun quickly turned into something much bigger, contributing to AnkiDroid made me realize how much I love working on open-source software. It's what cemented my decision to go all-in on Android development.

Regardless of whether I get selected or not, I'm already happy knowing that I've been able to give back to a project that once helped me. Being part of the AnkiDroid community; fixing issues, discussing features, and helping out newer contributors has been incredibly rewarding. It's also made me realize how important it is to keep the open-source spirit alive.

One day, I'd love to come full circle and mentor a GSoC contributor myself, not just to give back, but to help keep the cycle going. Open source thrives on passionate contributors lifting each other up, and I hope to play a role in that, both now and in the future.